

LAB 1 QUESTIONS

1. Create a **Student** class with the attributes **Student ID**, **Student Name**, and **Faculty**. Implement both a **default constructor** that initializes the object with predefined values and a **parameterized constructor** that accepts values from the user. Create a **Display()** method to display all the student information and demonstrate both constructors in the **Main()** method.
2. Create a **Rectangle** class with **Length** and **Width** as data members. Implement a default constructor and a parameterized constructor to initialize the object. Write methods to calculate the **Area** and **Perimeter** of the rectangle and display the calculated values using a separate display method.
3. Create a base class **Person** containing the attributes **Name** and **Address**. Derive an **Employee** class from the **Person** class by adding **Employee ID** and **Salary**. Write methods to input and display all employee information by utilizing the inherited properties from the base class.
4. Develop a **College Management System** by creating a base class named **Person** with common attributes such as **Name** and **Age**. Derive two classes named **Student** and **Teacher** from the base class. The **Student** class should contain **Roll Number** and **Faculty**, whereas the **Teacher** class should contain **Subject** and **Salary**. Create objects of both derived classes and display their complete information.
5. Create a **Calculator** class and overload the **Add()** method to perform addition of two integers, two double values, and three integers. Call each overloaded method from the **Main()** method and display the results to demonstrate compile-time polymorphism.
6. Create a class named **AreaCalculator** and overload the **Area()** method to calculate the area of a **Circle**, **Rectangle**, and **Triangle** by accepting different sets of parameters. Display the calculated area for each shape using appropriate method calls.
7. Create a base class named **Animal** containing a virtual method called **MakeSound()**. Derive three classes named **Dog**, **Cat**, and **Cow** from the **Animal** class and override the **MakeSound()** method in each class to display the appropriate sound of each animal. Demonstrate runtime polymorphism by calling the overridden methods through a base class reference.
8. Create a base class named **Shape** with a virtual method called **Draw()**. Derive the classes **Circle**, **Rectangle**, and **Triangle**, and override the **Draw()** method in each class to display an appropriate message indicating which shape is being drawn. Demonstrate method overriding by invoking the method using base class references.

LAB 1 QUESTIONS

9. Create an abstract class named **Vehicle** containing the attributes **Vehicle Number** and **Brand**, along with an abstract method named **Start()**. Create derived classes named **Car** and **Bike** that implement the **Start()** method. Instantiate the derived classes and display their respective start messages.
10. Design an abstract class named **BankAccount** containing the data members **Account Number**, **Account Holder Name**, and **Balance**, along with an abstract method named **CalculateInterest()**. Derive two classes named **SavingAccount** and **CurrentAccount**, implement the abstract method according to each account type, and display the calculated interest.
11. Create an interface named **IPrintable** containing a method called **Print()**. Implement this interface in two classes named **Student** and **Employee**. Store appropriate information in each class and use the implemented **Print()** method to display the details of both objects.
12. Create two interfaces named **ICamera** and **IMusicPlayer**, where the first interface contains the method **TakePhoto()** and the second contains the method **PlayMusic()**. Implement both interfaces in a class named **SmartPhone** and demonstrate multiple interface implementation by calling both methods from the **Main()** method.
13. Develop a **Hospital Management System** that demonstrates the concepts of **Constructors**, **Inheritance**, **Method Overloading**, **Method Overriding**, **Abstract Classes**, and **Interfaces**. Create an abstract class named **Hospital**, derive **Doctor** and **SpecialistDoctor** classes from it, and implement an interface named **IAppointment** containing methods for booking and canceling appointments. Overload the **CalculateBill()** method to calculate different types of bills and override the **DisplayDetails()** method to display doctor information. Create appropriate objects to demonstrate all the required object-oriented programming concepts in a single application.
14. Create a program that demonstrates the use of **Partial Classes** in C#. Divide a class named **Student** into two separate files using the **partial** keyword. In one file, define the data members **Student ID** and **Student Name**, and in the other file, implement methods to input and display the student information. Create an object of the class in the **Main()** method and demonstrate that both parts behave as a single class.
15. Create a **Library** management program using **Partial Classes**. Split the **Book** class into two files. In the first file, declare the properties **Book ID**, **Title**, and **Author**, and in the second file, implement methods to issue a book, return a book, and display book details. Demonstrate the functionality by creating objects and calling the appropriate methods.
16. Create a C# program that demonstrates the use of a **Single-Cast Delegate**. Define a delegate that can reference a method for calculating the square of a number. Accept a number from the user, invoke the delegate, and display the calculated square.

LAB 1 QUESTIONS

17. Develop a program to demonstrate the use of a **Multicast Delegate**. Create two methods named DisplayWelcome() and DisplayDateTime(). Create a delegate that invokes both methods sequentially and display the output to demonstrate multicast delegate functionality.
18. Create a **Student Result Processing System** using delegates. Define separate methods to calculate the **Total Marks**, **Percentage**, and **Grade** of a student. Use delegates to invoke these methods and display the complete result of the student.
19. Create a C# program that demonstrates the use of **Anonymous Methods** and **Lambda Expressions** with delegates. Write a delegate that performs the addition of two numbers and implement it first using an anonymous method and then using a lambda expression. Display the results of both implementations.
20. Create a class named **StudentCollection** that stores the names of five students in an array. Implement an **Indexer** to access and modify student names using array-like syntax. Demonstrate adding, retrieving, and updating student names through the indexer.
21. Create a **BookCollection** class that stores details of multiple books using an array or list. Implement an **Indexer** to access book titles by their index position. Display all stored book titles and allow the user to search for a book using its index.
22. Develop a **Student Management System** that demonstrates the concepts of **Partial Classes**, **Delegates**, and **Indexers**. Split the **Student** class into multiple files using partial classes, use delegates to calculate the student's total marks, percentage, and grade, and implement an indexer to store and access multiple student records. The program should allow users to add, view, update, and process student information while demonstrating all three concepts in a single C# Console Application.